

A Multi-Objective Optimization Framework for Robots Design

Ghadi Nehme

Department of Mechanical Engineering
Stanford University

ghadi@stanford.edu

Abstract

The rapid advancement in robotics technology has extended its application in diverse fields requiring specialized operational tasks, ranging from industrial automation to surgical interventions. Designing robots that efficiently perform specific tasks while navigating complex operational environments poses significant challenges. This project introduces a robust multi-objective optimization framework designed to fine-tune robot parameters for task-specific effectiveness, balancing critical attributes such as efficiency, operational safety, and dexterity. The framework incorporates advanced computational techniques including genetic algorithms and gradient descent, focusing on optimizing metrics such as manipulability, error minimization, and structural compactness while ensuring compliance with environmental and safety constraints. To demonstrate the applicability and effectiveness of this framework, case studies involving simple test scenarios, a dental robot for precise operations within constrained environments, and a surgical robot optimized for complex medical procedures are discussed.

1. Introduction

The field of robotics has seen substantial growth, driven by technological advancements and an increasing demand for automation across various sectors. Traditionally, robot design has been a compromise between multiple competing factors including cost, efficiency, and adaptability to diverse operational contexts. The emergence of sophisticated applications, particularly in precision-demanding fields like surgery and micro-manufacturing, requires robots that are not only mechanically proficient but also optimized for specific tasks.

The complexity of modern robotic systems and their operational environments necessitates a design approach that transcends traditional engineering techniques. Multi-objective optimization provides a strategic framework to address this, allowing for the simultaneous optimization of

several design criteria that often conflict with one another. For instance, increasing a robot's range of motion might inadvertently reduce its precision or increase its susceptibility to errors.

This project details the development and application of a multi-objective optimization framework that integrates both theoretical and computational methods to derive optimal robot designs. The framework employs genetic algorithms to explore a wide solution space effectively and gradient descent methods to fine-tune the solutions, ensuring that the final robot design meets predefined performance metrics such as minimal error rates, high manipulability, and optimal use of space. The source code is available [here](#).

2. Related Work

Prior works have delved into the challenges and methodologies associated with optimizing robot design for specific tasks and environments. One influential study, "Realization of Task-based Designs Involving DH Parameters: A Modular Approach" [1], introduces a modular strategy for developing task-based serial manipulators. This approach utilizes reconfigurable modules defined by Denavit-Hartenberg (DH) parameters to allow for the adaptation of robot structures to varied tasks in cluttered environments.

Additionally, "A Multi-objective Optimization Design of Industrial Robot Arms" [2], explores multi-objective optimization techniques to balance various design criteria such as mechanical efficiency and cost-effectiveness in industrial robot arms. This work provides foundational insights into the trade-offs involved in optimizing robot design, informing approaches that consider multiple conflicting objectives.

Lastly, "Shape Design Optimization of a Robot Arm Using a Surrogate-Based Evolutionary Approach" [3] presents an evolutionary algorithm-based method that employs surrogate models to efficiently optimize the shape of robot arms. This approach addresses the challenge of high computational costs associated with direct optimization methods and demonstrates the potential of surrogate models in speeding up the optimization process while maintaining accuracy.

3. Approach

3.1. Robot Definition

A robot can be formally defined as a tuple

$$\mathcal{R} = (\mathcal{L}, \mathcal{J}),$$

where $\mathcal{L}$ represents the links and $\mathcal{J}$ represents the joints of the robot. This representation is commonly detailed using the Denavit-Hartenberg (DH) parameters, which standardize the way to express the kinematic configurations of robot manipulators. The parameters are used to describe the relative transformations between consecutive links. We will be using a representation equivalent to the Denavit-Hartenberg (DH) parameters. A robot will be represented as a sequence of joints and links associated with its corresponding parameters shown in Table 1.

Joint Type	Joint and Link Parameter
Revolute	$\mathbf{d}_1$: Axis of rotation of the joint $\mathbf{d}_2$: Axis of the link a : Link length
Prismatic	$\mathbf{d}_1$: Axis of the joint a : Link length $d_{\max}$: Maximal length of the link
Fixed	$\mathbf{d}_1$: Axis of rotation of the joint $\mathbf{d}_2$: Axis of the link a : Link length α : Link twist

Table 1. Robot Link Parameters

Here is an example:

$$\mathcal{R} = \{[\text{Fixed: } \mathbf{d}_1 = e_y, \mathbf{d}_2 = e_z, a = 2, \alpha = \pi/2], \\ [\text{Prismatic: } \mathbf{d}_1 = e_x, a = 1, d_{\max} = 5], \\ [\text{Revolute: } \mathbf{d}_1 = e_x, \mathbf{d}_2 = e_y, a = 1], \\ [\text{Revolute: } \mathbf{d}_1 = e_z, \mathbf{d}_2 = e_x, a = 1]]\}$$

For simplicity, we will rewrite this in the following way:

$$\mathcal{R} = \{[2, 1, 2, 2, \pi/2], \\ [1, 0, 1, 5], \\ [0, 0, 1, 1], \\ [0, 2, 0, 1]]\},$$

where the joint type is a discrete variable that takes the value 0, 1 and 2 for a revolute, prismatic and fixed joint respectively, and the axis is a discrete variable that takes the values 0, 1 and 2 for e_x , e_y and e_z respectively.

In conjunction with the theoretical framework, we developed a robot simulator (Fig. 1) to visually and functionally represent the robot's configuration and behavior in a

controlled environment. This simulator constructs the robot based on a set of parameters including the base position, orientation basis, and a sequence of links defined by the parameters in Table 1. Each link, whether revolute, prismatic, or fixed, is accurately rendered with respect to its kinematic dependencies and interactions, using transformation matrices to compute their respective positions and orientations in 3D space.

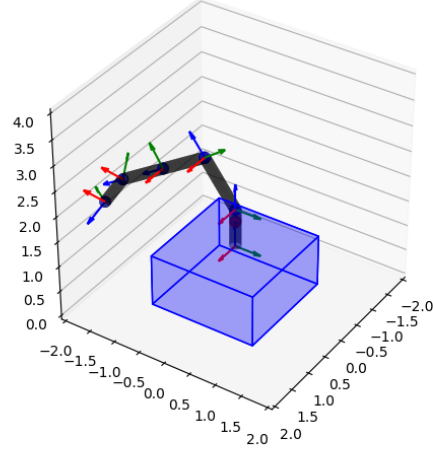


Figure 1. Robot simulator

The simulator features advanced functionalities such as collision detection, both with environmental objects and among the robot's own components, and kinematic analyses like forward kinematics, Jacobian computation, and singularity detection. Visualization tools within the simulator allow for dynamic representation of the robot's motion and interaction with its environment, enabling real-time adjustments and optimizations.

3.2. Task Definition

The task associated with a robot is defined as

$$\mathcal{T} = (\mathcal{E}, \mathcal{F}),$$

where $\mathcal{E}$ represents the environment constraints and $\mathcal{F}$ is the function of performance. The environment $\mathcal{E}$ includes physical or operational boundaries such as the boundaries of the mouth for a dental surgery robot, as well as the desired trajectories for the robot's end effector, denoted by $\mathcal{P} = \{\mathbf{x}_i\}_{i=1}^N$, where N is the number of points in the trajectory. The function $\mathcal{F}$ is a linear combination of multiple performance metrics, which we aim to optimize:

3.2.1 Trajectory Error

We aim to minimize the error between the end effector position and the trajectory points associated with the task, denoted by e . To compute this error, we define an objective

function $g(\theta, x_d)$ which returns the L2-norm between the desired position x_d on the task trajectory and the position of the end effector, which we obtain by doing forward kinematics using the control inputs θ . Then, we get the optimal control inputs by minimizing the objective function g :

$$\theta_{opt,d} = \arg \min_{\theta} g(\theta, x_d)$$

And we obtain e as the mean of g evaluated for each point $x_i \in \mathcal{P}$:

$$e = \frac{1}{N} \sum_{i=1}^N g(\theta_{opt,i}, x_i)$$

3.2.2 Design Compactness

A compact design helps in reducing material costs and improves the robot's aesthetic integration into human-centric environments. To achieve a compact robot design, we want to minimize the L1 or L2 norm of the lengths of the robot's links, denoted by L_1 and L_2 respectively. Thus, we have:

$$L_1 = \sum_{\ell \in \mathcal{L}} |\ell| \text{ and } L_2 = \sum_{\ell \in \mathcal{L}} |\ell|^2$$

3.2.3 Degrees of freedom

The degrees of freedom of a robotic arm is equal to its number of joints n . We aim to minimize n since a high degree of freedom robot tends to increase the complexity of the control as well as the total cost.

3.2.4 Manipulability Index

In robotics, the manipulability index is a measure of the ability of a robotic manipulator to move and exert forces in various directions from a given pose. We want to maximize this index to enhance the dexterity and flexibility of the robotic arm. The manipulability index is defined as

$$M = \sqrt{\det(JJ^T)},$$

where J is the Jacobian matrix of the robot. The Jacobian matrix, defined as

$$J(\theta) = \frac{\partial \mathbf{p}(\theta)}{\partial \theta}$$

where $\mathbf{p}(\theta)$ represents the position of the end effector given control inputs θ and θ represents the vector of joint parameters (angle for revolute joint or displacement for prismatic joint), quantifies how changes in joint parameters affect the position and orientation of the end effector.

In this framework, the manipulability index will be the minimal value achieved for the different points in the trajectory:

$$M = \min_i \sqrt{\det(J(\theta_{opt,i})J(\theta_{opt,i})^T)}, \text{ with } i \in \{1, \dots, N\}$$

3.2.5 Singularity Index

In robotics, a singularity occurs when the Jacobian matrix J of a robot becomes singular, meaning it loses full rank and some of its degrees of freedom.

To avoid such issues, we will maximize the smallest singular value $\sigma_n(\theta_{opt,i})$ of the Jacobian matrix, which will insure that we are as far away as possible from the Jacobian matrix being singular. In this framework, the singularity index will be the minimal value achieved for the different points in the trajectory:

$$S = \min_i \sigma_n(\theta_{opt,i}), \text{ with } i \in \{1, \dots, N\}$$

3.2.6 Self-Collision and Environmental Collision Avoidance

Collision avoidance is a critical factor in ensuring the safety and integrity of the robot. The simulator actively checks for self-collisions between non-consecutive links of the robot as it navigates through its operational space. Environmental collision avoidance is handled by simulating potential interactions between the robot and its operating environment. The robot's kinematic configurations are tested against a predefined set of environmental constraints, ensuring that no part of the robot invades restricted spaces. In this framework, the number of self-collisions for a specific trajectory is given by:

$$C_S = \sum_{i=1}^N 1\{\text{Self-collision when } \theta = \theta_{opt,i}\}$$

and the number of collisions with the environment for a specific trajectory is given by:

$$C_E = \sum_{i=1}^N 1\{\text{Environment collision when } \theta = \theta_{opt,i}\}$$

3.3. Multi-Objective Optimization Problem

The primary objective of our framework is formulated as an optimization problem:

$$\min_{\mathcal{R}} \mathcal{F},$$

where $\mathcal{R}$ denotes the robot configuration and $\mathcal{F}$ is the performance function derived from the task definition $\mathcal{T}$. This formulation seeks to find the optimal set of robot parameters that satisfy the predefined metrics of performance while adhering to environmental and operational constraints. To address the challenge of optimizing multiple conflicting objectives within our robot design framework, we use the weighted sum method. We can thus rewrite the problem

in the following way:

$$\begin{aligned} \min_{\mathcal{R}} \quad & \alpha_e e + \alpha_n n + \alpha_1 L_1 + \alpha_2 L_2 \\ \text{subject to} \quad & C_S = 0 \\ & C_E = 0 \\ & S \geq S_0 \\ & M \geq M_0 \end{aligned}$$

with $\alpha_e, \alpha_n, \alpha_1, \alpha_2 \geq 0$ and S_0 and M_0 the minimal values required for S and M respectively.

4. Method

In this section, we present the method we used to solve the optimization problem defined in section 2.3.

4.1. Penalty Method

To handle the constraints in our optimization problem, we apply the penalty method by augmenting the objective function with penalty terms for constraint violations:

$$\alpha_C(C_E + C_S),$$

where α_C is a large coefficient, ensuring that any feasible solution must satisfy $C_E = 0$ and $C_S = 0$. Similarly, penalties are added for the singularity and manipulability constraints to encourage solutions that not only satisfy but exceed the thresholds S_0 and M_0 .

Thus, the unconstrained objective function becomes:

$$\begin{aligned} \mathcal{F} = & \alpha_e e + \alpha_n n + \alpha_1 L_1 + \alpha_2 L_2 + \alpha_C(C_E + C_S) \\ & + \alpha_S \max(S_0 - S, 0) + \alpha_M \max(M_0 - M, 0), \end{aligned}$$

with $\alpha_C, \alpha_S, \alpha_M$ large coefficients.

4.2. Optimization Algorithm

The optimization process begins with a genetic algorithm to find an optimal configuration of the joints and the links, followed by a gradient descent to fine-tune the continuous variables, specifically the lengths of the links. This method ensures a comprehensive exploration of the solution space followed by precise refinement, optimizing the robot design for performance, compactness, and operational safety.

4.2.1 Genetic Algorithm

The robot configuration involves a mixture of discrete and continuous variables, where joint types, axis (discrete), and link lengths (continuous) must be optimized simultaneously. Genetic algorithms are ideal for handling such mixed-variable problems due to their capability to explore a vast and heterogeneous search space effectively, avoiding local minima and providing diverse candidate solutions. Key parameters include population size, mutation rate, and crossover probability. The pseudocode is as follows:

Algorithm 1: Genetic Algorithm for Robot Optimization

```

Initialize population with random configurations
for generation = 1 to  $N_{\text{generations}}$  do
    Evaluate fitness of each individual
    Select parents via tournament selection
    Apply crossover and mutation to create new offspring
    Replace least fit individuals with new offspring
end while

```

4.2.2 Gradient Descent with Line Search

After obtaining a near-optimal solution from the genetic algorithm, we apply gradient descent with line search to refine the solution by optimizing the link lengths. The pseudocode for this phase is:

Algorithm 2: Gradient Descent with Line Search to optimize the continuous variables of $\mathcal{R}$

```

Data: learning rate  $\alpha$ , decay rate  $\gamma$ , maximum steps
 $N$ , indices of continuous variables for
optimization  $I$ , best solution from genetic
algorithm  $x_{\text{best}}$ , objective function  $f$ 
Initialize  $x \leftarrow x_{\text{best}}$ 
for  $i = 1$  to  $N$  do
    Calculate numerical gradient  $g \leftarrow \nabla f(x)$ 
    if  $\|g\| < 10^{-5}$  then
        Break
    end if
    Normalize the gradient  $g \leftarrow g/\|g\|$ 
    Copy previous state  $x_{\text{prev}} \leftarrow x$ 
    Evaluate previous function value  $f_{\text{prev}} \leftarrow f(x)$ 
    Initialize  $f_0 \leftarrow \infty$ 
    while  $f_0 > f_{\text{prev}}$  do
         $x \leftarrow x_{\text{prev}}$ 
        for  $\text{idx}$  in indices do
            Update  $x[\text{idx}] \leftarrow \max(x[\text{idx}] - \alpha g[\text{idx}], 0)$ 
        end for
        Evaluate current function value  $f_0 \leftarrow f(x)$ 
        Reduce learning rate  $\alpha \leftarrow \gamma \alpha$ 
    end while
end for
return  $x$ 

```

5. Experiments, Results, and Discussion

This section presents three case studies to evaluate the effectiveness and practical applicability of our multi-objective optimization framework for robot design. The case studies include a simple test example, a dental robot application, and a surgical robot application. These examples highlight

Table 2. Comparative Results of Optimization Metrics Across Three Case Studies

Case Study	n	Self-collisions	Env. collisions	Manipulability Index	Singularity Index	Error
Simple Test Case	1	0	0	0	1	10^{-6}
Dental Robot	6	0	0	58.28	0.82	10^{-6}
Surgical Robot	7	0	0	82.11	1.53	10^{-6}

the framework’s capabilities in optimizing robot designs under varying constraints and requirements. All the hyperparameters of these case studies can be found in the [Appendix](#). The videos of the robots following the trajectory are available [here](#). We end this section with an analysis of the Pareto front when optimizing the number of degrees of freedom and the L2-norm of the links.

5.1. Simple Test Case

To validate our multi-objective optimization framework, we initiated a straightforward test where the desired trajectory was a straight line along the z -direction. Theoretically, a prismatic joint aligned with the z -axis represents the optimal solution for such a trajectory, offering direct and efficient movement.

We configured the optimization without environmental constraints to focus on trajectory achievement. The aim was to verify if the optimization would autonomously identify and implement the optimal solution of a prismatic joint along the z -axis.

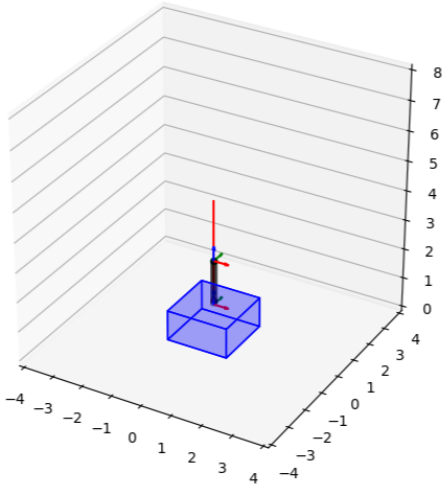


Figure 2. Visualization of the solution found for the simple test case.

The solution obtained (Figure 2) confirmed the framework’s robustness; the optimization correctly converged to

the expected solution of a prismatic joint aligned with the z -axis. We observe in Table 2 that the manipulability index is 0 which is expectable since it is a 1 dof robot. This outcome not only validates our framework against theoretical expectations but also underscores its precision in configuring effective robot designs under specified conditions.

5.2. Dental Robot Application

In this case study, we tailored our multi-objective optimization framework to design a dental robot capable of operating within the complex confines of a human mouth. A series of trajectories typically used in dental surgeries were selected to define the robot’s desired trajectory. Additionally, constraints mimicking the spatial limitations of a patient’s mouth were incorporated to ensure realistic and practical robot movement, as illustrated in Figure 3.

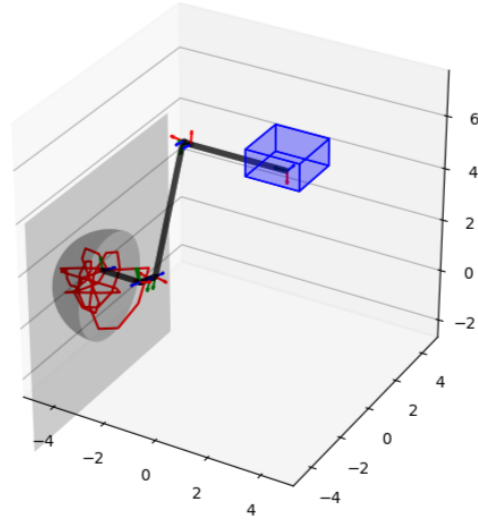


Figure 3. Optimized dental robot navigating within the constraints similar to a patient’s mouth. In grey, we can see the boundary of the constraint. We can also see the plot of the task trajectory in red.

The results, detailed in Table 2, confirmed the success of the optimization process. The robot configuration achieved zero collisions and nearly zero operational error. Both the

singularity index and manipulability index were above the set thresholds, demonstrating the framework’s effectiveness in optimizing for precision and safety in restricted environments.

5.3. Surgical Robot Application

For the surgical robot case study, our framework was applied to optimize a robotic system designed for minimally invasive surgeries. The operational requirements were defined based on a series of trajectories that are commonly used in such surgical procedures. Constraints reflecting the limited workspace typical of surgical environments were incorporated, as depicted in Figure 5.

The results were highly successful, as outlined in Table 2. The surgical robot achieved zero collisions and nearly zero error in trajectory following. Additionally, both the singularity and manipulability indices exceeded the predefined safety and efficiency thresholds, confirming the effectiveness of the optimization framework in designing robots that are both precise and reliable for complex surgical tasks.

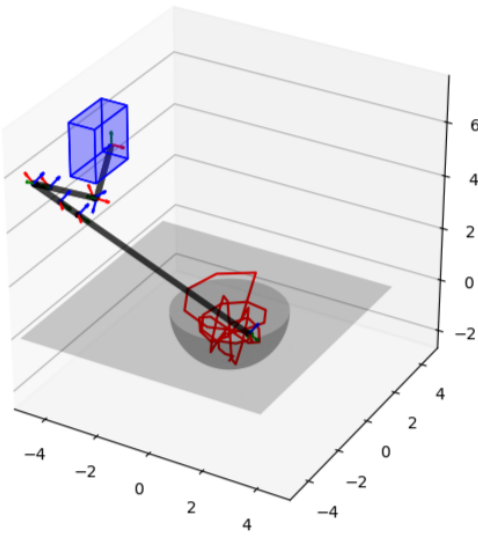


Figure 5. Optimized surgical robot designed to operate within the constrained surgical workspace. In grey, we can see the boundary of the constraint. We can also see the plot of the task trajectory in red.

5.4. Analysis of the Pareto Front between L2 Norm and Degrees of Freedom

The trade-offs between the compactness of a robot’s design and its flexibility, represented by the number of degrees of freedom n , are crucial in optimizing robot designs for specific applications. To visualize and analyze these trade-offs, we employ a Pareto front analysis, which helps in un-

derstanding the relationship between the L2 norm of the link lengths and the number of degrees of freedom.

The Pareto front shown in Figure 6 represents optimal trade-offs where no objective can be improved without worsening the other. Each point on the Pareto front corresponds to a robot configuration where the L2 norm of the link lengths and the number of degrees of freedom are optimized to a degree that any further improvement in one would lead to a compromise in the other.

As the number of degrees of freedom n increases, we generally observe a decrease in the L2 norm, indicating shorter link lengths. This is intuitive as more joints allow for a more compact folding of the robot structure, thus reducing the overall length of the links required to span a particular operational space, which can be observed in Figure 4, since all solutions have very straight shape even if it has many degrees of freedom.

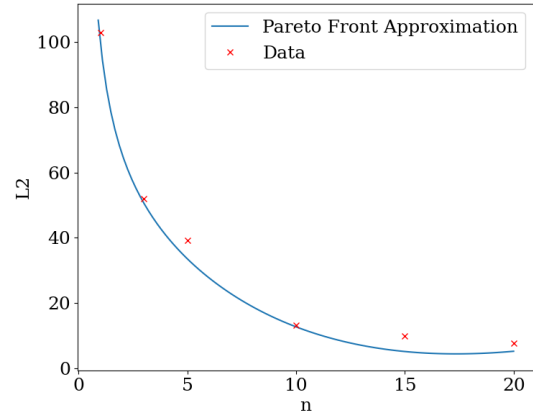


Figure 6. Pareto front showing the trade-off between the L2 norm of link lengths and the number of degrees of freedom n . Each point represents a feasible robot design that balances compactness and flexibility.

This analysis is vital for making informed decisions in robot design, particularly when considering specific application constraints. For instance, in applications requiring highly compact robots, such as surgical or wearable robots, a higher number of joints may be preferable despite the potential increase in control complexity. Conversely, in applications where simplicity and robustness are paramount, minimizing the number of degrees of freedom could be more critical, even at the cost of slightly larger robots.

By applying multi-objective optimization and analyzing the Pareto front, we can systematically explore and select robot configurations that best meet the specific needs of various applications, ensuring an optimal balance between compactness and flexibility.

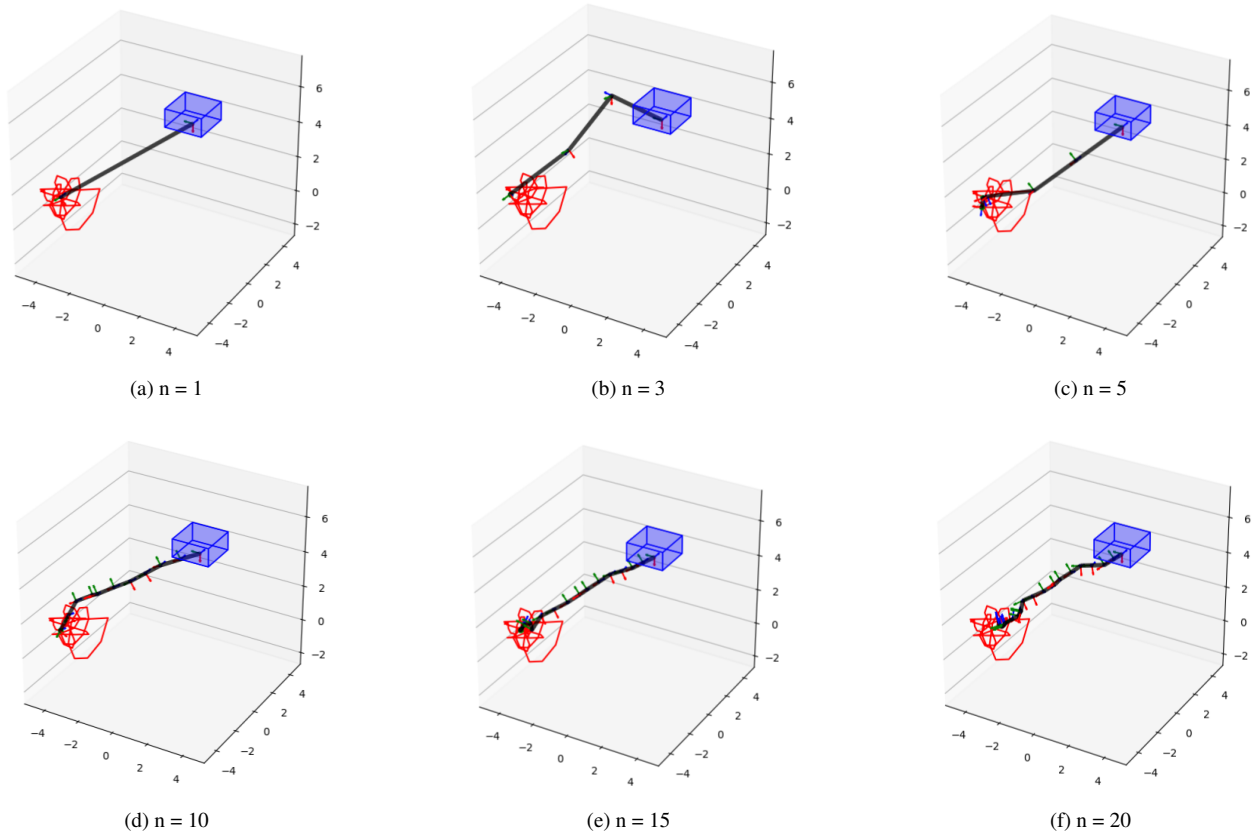


Figure 4. The solutions obtained when optimizing the objective function with a fixed degree of freedom n .

6. Conclusion

We presented a multi-objective optimization framework for robotic design, demonstrating its effectiveness through various case studies including simple tests, dental robotics, and surgical applications. The framework successfully optimized robotic configurations to meet specific operational needs while balancing critical performance metrics such as manipulability, singularity index, and error minimization. The application of advanced optimization techniques, including genetic algorithms and gradient descent, allowed for the precise and efficient tuning of design parameters under complex operational constraints.

Looking ahead, there are several avenues for extending this project to enhance its scope and impact further:

- **Optimization Across Robotic Platforms:** Future work could include extending the framework to optimize systems comprising multiple robotic arms as well as other types of robotic systems, such as quadruped and biped robots.
- **Scalability with Enhanced Computation:** With access to greater computational resources, the framework could handle optimizations over a larger set of tasks.

This scalability would facilitate the development of more versatile and robust robotic arms capable of performing a diverse array of tasks.

- **Incorporating Additional Objectives:** Further research could also explore the inclusion of additional objectives in the optimization process, such as those related to the control performance and the mass of the robot. This would help in designing robots that are not only efficient in task performance but also optimized for energy consumption and dynamic response.

References

- [1] Satwinder Singh and Ekta Singla. Realization of task-based designs involving dh parameters: a modular approach. *Intelligent Service Robotics*, 9:289–296, 2016. 1
- [2] Sallam A Kouritem, Mohammed I Abouheaf, Nabil Nahas, and Mohamed Hassan. A multi-objective optimization design of industrial robot arms. *Alexandria Engineering Journal*, 61(12):12847–12867, 2022. 1
- [3] JC Hsiao, Kumar Shivam, CL Chou, and TY Kam. Shape design optimization of a robot arm using a surrogate-based evolutionary approach. *Applied Sciences*, 10(7):2223, 2020. 1

Appendix

Here are the different hyperparameters used in the different optimizations:

I. Simple Test Case

- Environmental constraints: No
- Number of generations: 5
- Population size: 200
- Mutation rate: 0.2
- Crossover probability: 0.5
- Tournament size: 5
- Allowed joint: $\{0, 1, 2\}$
- Constraints for n : $n_{min} = 1, n_{max} = 7$
- $\alpha_e = 10000$
- $\alpha_n = 1$
- $\alpha_1 = 1$
- $\alpha_2 = 0$
- $\alpha_C = 1000$
- $\alpha_S = 1000$
- $\alpha_M = 1000$
- $S_0 = 0.5$
- $M_0 = 5$

II. Dental Robot Application

- Environmental constraints: Yes
- Number of generations: 10
- Population size: 100
- Mutation rate: 0.2
- Crossover probability: 0.5
- Tournament size: 3
- Allowed joint: $\{0, 2\}$
- Constraints for n : $n_{min} = 6, n_{max} = 12$
- $\alpha_e = 10000$
- $\alpha_n = 1$
- $\alpha_1 = 0$

- $\alpha_2 = 1$
- $\alpha_C = 1000$
- $\alpha_S = 1000$
- $\alpha_M = 1000$
- $S_0 = 0.5$
- $M_0 = 5$

III. Surgical Robot Application

- Environmental constraints: Yes
- Number of generations: 10
- Population size: 100
- Mutation rate: 0.2
- Crossover probability: 0.5
- Tournament size: 3
- Allowed joint: $\{0, 1, 2\}$
- Constraints for n : $n_{min} = 6, n_{max} = 12$
- $\alpha_e = 10000$
- $\alpha_n = 1$
- $\alpha_1 = 1$
- $\alpha_2 = 0$
- $\alpha_C = 1000$
- $\alpha_S = 1000$
- $\alpha_M = 1000$
- $S_0 = 0.5$
- $M_0 = 5$